

TUTORIAL COMPLETO DE EXECUÇÃO COM DOCKER E KUBERNETES

Aplicação Biblioteca em três provedores distintos

AWS (Amazon EKS) + Google Cloud (GKE Autopilot) + DigitalOcean VPS (K3s)

Faculdade Alfa Umuarama

Sistemas para Internet

Acadêmicos: Gabrielle Bonini RA: 14506

Guilherme Moraes RA: 5606

Felype Augusto RA: 14537

Larissa Fernanda RA: 14536

Professor(a): Diogo Ranghetti

Objetivo e escopo

Este documento apresenta um roteiro reproduzível para containerizar a aplicação web Biblioteca com Docker e executar a mesma aplicação em Kubernetes em três ambientes distintos: Amazon Web Services, Google Cloud e uma VPS DigitalOcean. O foco é acadêmico: demonstrar a portabilidade da mesma imagem, o uso de objetos Kubernetes equivalentes, a validação funcional e a comparação de custos e esforço operacional.

Escolha dos ambientes: A combinação AWS + Google Cloud + VPS atende à regra de usar dois grandes provedores de nuvem e um terceiro ambiente diferente, sem repetir o mesmo provedor.

Limite deste documento: O tutorial fornece comandos, arquivos e validações esperadas. As capturas de tela e evidências reais de execução devem ser produzidas pela equipe ao executar os passos em suas próprias contas; este documento não inventa evidências de recursos que não foram provisionados.

Sumário do tutorial

- 1. Regra dos três ambientes e configuração escolhida
- 2. Análise da aplicação Biblioteca
- 3. Arquitetura mínima da solução
- 4. Pré-requisitos e organização dos arquivos
- 5. Obtenção e verificação do código-fonte
- 6. Containerização com Docker
- 7. Teste local da imagem
- 8. Manifestos Kubernetes comuns
- 9. Provedor 1 - AWS com Amazon EKS e ECR
- 10. Provedor 2 - Google Cloud com GKE Autopilot e Artifact Registry
- 11. Provedor 3 - DigitalOcean VPS com K3s
- 12. Validação funcional e evidências
- 13. Diagnóstico e solução de erros comuns
- 14. Comparação de custos
- 15. Comparação qualitativa
- 16. Recomendação final
- 17. Limpeza dos recursos
- 18. Referências oficiais

1. Regra dos três ambientes e configuração escolhida

Ambiente	Produto	Região	Configuração acadêmica
AWS	Amazon EKS + ECR	us-east-1 (N. Virginia)	1 node gerenciado t3.medium (2 vCPU, 4 GiB), Service LoadBalancer
Google Cloud	GKE Autopilot + Artifact Registry	us-central1 (Iowa)	1 Pod da aplicação; recursos sob demanda gerenciados pelo Autopilot; Service LoadBalancer
DigitalOcean	Droplet VPS + K3s	nyc3 (ou região disponível próxima)	Droplet Basic 2 vCPU / 4 GiB / 80 GiB; cluster K3s de nó único; Service NodePort 30080

Justificativa: AWS e Google Cloud oferecem Kubernetes gerenciado e integração nativa com registro, rede e observabilidade. A DigitalOcean foi escolhida como terceiro provedor por permitir uma VPS Linux simples, onde o Kubernetes é instalado pela própria equipe via K3s. Essa diferença evidencia o contraste entre serviço gerenciado e responsabilidade operacional própria.

Comparabilidade: As regiões escolhidas são regiões dos Estados Unidos para reduzir variações de preço e simplificar o laboratório. A equipe pode alterar as regiões, mas deve registrar a região realmente utilizada e recalculando os custos.

2. Análise da aplicação Biblioteca

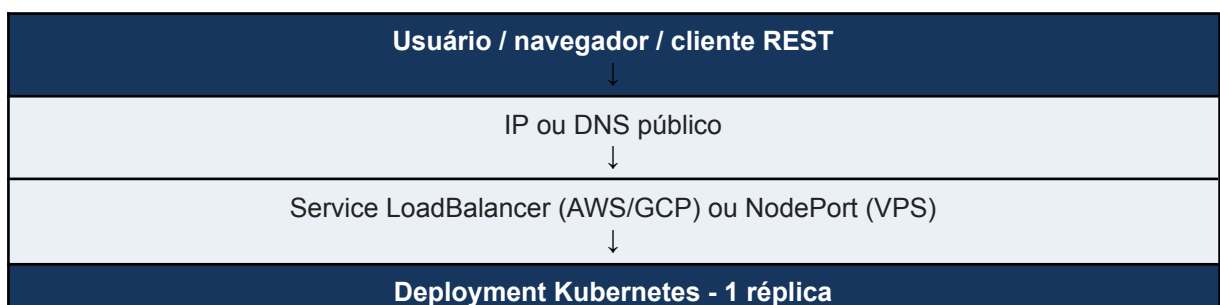
Item	Característica
Framework	Spring Boot 4.0.6
Java	Java 21
Interface web	Thymeleaf com login por formulário
API	REST em /api/**, protegida por JWT Bearer Token
Persistência	Spring Data JPA + H2
Banco	H2 em memória: jdbc:h2:mem:banco
Porta	8080
Build	Maven Wrapper / Maven
Usuário administrador	admin / admin (somente laboratório)
Usuário comum	user / user (somente laboratório)

A interface web permite autenticação e operações de cadastro/consulta de autores, livros e usuários conforme permissões. A API REST expõe autenticação em /api/auth/login e exige o cabeçalho Authorization: Bearer <token> nas demais rotas protegidas.

Limitação crítica do H2: O banco está em memória dentro do processo Java. Ao reiniciar o container ou o Pod, os dados cadastrados durante a execução são perdidos. Por isso, este tutorial usa replicas: 1 e classifica a implantação como demonstrativa. Em produção, seria recomendável migrar para um banco persistente externo.

Segurança: As credenciais admin/admin e user/user são adequadas apenas ao exercício. Não use credenciais padrão em produção. O segredo JWT também não deve ser gravado na imagem, no repositório ou em capturas de tela.

3. Arquitetura mínima da solução



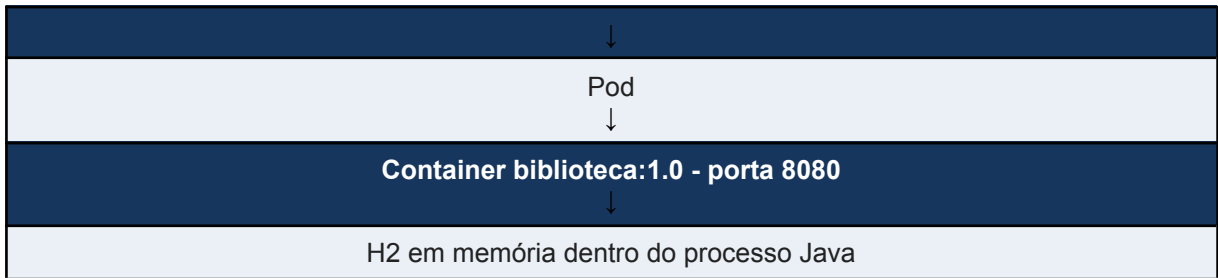


Figura textual 1 - Fluxo mínimo da implantação. A mesma aplicação e a mesma imagem lógica são utilizadas nos três ambientes.

4. Pré-requisitos e organização dos arquivos

- Docker Engine ou Docker Desktop em funcionamento.
- Git para clonar o projeto.
- kubectl instalado.
- Editor de texto ou IDE, como VS Code ou IntelliJ IDEA.
- Conta AWS com permissões para IAM/EKS/ECR/EC2/VPC/ELB.
- AWS CLI e eksctl instalados para o ambiente AWS.
- Conta Google Cloud com projeto e faturamento habilitados.
- Google Cloud CLI (gcloud) para o ambiente GCP.
- Conta DigitalOcean, chave SSH e acesso administrativo à VPS.
- Cliente SSH no computador local.

Estrutura sugerida depois da preparação:

```

biblioteca/
├── Dockerfile
├── .dockerignore
├── pom.xml
├── mvnw
├── mvnw.cmd
├── src/
├── k8s/
│   ├── secret-exemplo.yaml
│   ├── deployment.yaml
│   ├── service-cloud.yaml
│   └── service-vps.yaml

```

5. Obtenção e verificação do código-fonte

Clonar o projeto:

```

git clone https://github.com/dpRanghetti/biblioteca.git
cd biblioteca

```

Registrar a versão utilizada:

```

git rev-parse HEAD
git log -1 --format="%h | %ad | %s" --date=iso

```

Antes de criar a imagem, confirme a presença de pom.xml, mvnw/mvnw.cmd, src/main/java e src/main/resources/application.properties. O commit ou a data da versão deve aparecer nas evidências do trabalho.

Verificações úteis:

```
# Linux/macOS
ls -la
find src/main -maxdepth 2 -type d | sort

# Windows PowerShell
Get-ChildItem
Get-ChildItem src\main -Recurse -Directory | Select-Object FullName
```

6. Containerização com Docker

6.1 Dockerfile multi-stage

Arquivo Dockerfile:

```
FROM maven:3.9-eclipse-temurin-21 AS build
WORKDIR /app
COPY pom.xml .
COPY src ./src
RUN mvn -B clean package -DskipTests

FROM eclipse-temurin:21-jre
WORKDIR /app
COPY --from=build /app/target/biblioteca-0.0.1-SNAPSHOT.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

O primeiro estágio contém Maven e o JDK para compilar a aplicação. O segundo estágio recebe apenas o JAR e um JRE Java 21, reduzindo componentes desnecessários na imagem final.

Se o nome do JAR mudar: Depois de executar o build, verifique a pasta target. Se o nome do arquivo for diferente de biblioteca-0.0.1-SNAPSHOT.jar, ajuste a linha COPY --from=build no Dockerfile.

6.2 .dockerignore

Arquivo .dockerignore:

```
.git
.idea
.vscode
target
docs
*.log
README.md
.env
*.pem
*.key
```

O dockerignore reduz o contexto enviado ao daemon do Docker, evita artefatos locais e ajuda a impedir o envio acidental de credenciais e chaves.

6.3 Construir e inspecionar a imagem

Build e verificação:

```
docker build -t biblioteca:1.0 .
docker image ls biblioteca
docker history biblioteca:1.0
docker image inspect biblioteca:1.0 --format "{{.Id}}"
```

Evidência: Registre a tag biblioteca:1.0, o ID/digest da imagem e uma captura do build concluído sem erros.

7. Teste local da imagem

Linux/macOS/Git Bash:

```
docker run --name biblioteca-local -p 8080:8080 \
-e API_SECURITY_TOKEN_SECRET="SUBSTITUA_POR_UM_SEGREDO_FORTE" \
biblioteca:1.0
```

Windows PowerShell:

```
docker run --name biblioteca-local -p 8080:8080 `
-e API_SECURITY_TOKEN_SECRET="SUBSTITUA_POR_UM_SEGREDO_FORTE" `
biblioteca:1.0
```

O Spring converte a propriedade api.security.token.secret para a variável de ambiente API_SECURITY_TOKEN_SECRET. Use um valor forte e não o publique.

Acompanhar logs:

```
docker logs -f biblioteca-local
```

Validação	Endereço / ação	Resultado esperado
Página inicial	http://localhost:8080/	Redireciona para login ou exibe a aplicação autenticada
Login	http://localhost:8080/login	Login admin/admin ou user/user funciona
H2 Console	http://localhost:8080/h2-console	Acesso administrativo; JDBC URL jdbc:h2:mem:banco
API login	POST http://localhost:8080/api/auth/login	Retorna token JWT para credenciais válidas

Exemplo de teste da autenticação REST:

```
curl -X POST http://localhost:8080/api/auth/login \
-H "Content-Type: application/json" \
-d '{"username":"admin","password":"admin"}'
```

Copie o token retornado e use-o no cabeçalho Authorization: Bearer <TOKEN> ao chamar uma rota protegida. O endpoint protegido específico pode ser escolhido após consultar as rotas do projeto.

Encerrar o teste local:

```
docker stop biblioteca-local
docker rm biblioteca-local
```

Regra prática: Não use o cluster de nuvem para depurar um Dockerfile que ainda não funciona localmente. Só prossiga quando a aplicação iniciar e o login funcionar.

8. Manifestos Kubernetes comuns

Os mesmos objetos lógicos são usados nos três ambientes. A alteração do caminho da imagem para o registro de cada provedor e a troca do Service LoadBalancer por NodePort na VPS são adaptações justificadas pelo ambiente.

8.1 Secret

k8s/secret-exemplo.yaml:

```
apiVersion: v1
kind: Secret
metadata:
  name: biblioteca-secret
type: Opaque
stringData:
  jwt-secret: "SUBSTITUIR-ANTES-DE-APLICAR"
```

Melhor prática para o laboratório: Não versione o secret com o valor real. Prefira criar o Secret diretamente no cluster com `kubectl create secret generic` e mantenha `secret-exemplo.yaml` apenas como modelo.

Criar o Secret sem arquivo real:

```
kubectl create secret generic biblioteca-secret \
  --from-literal=jwt-secret="VALOR_FORTE_E_NAO_PUBLICADO"
```

8.2 Deployment

k8s/deployment.yaml:

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: biblioteca
spec:
  replicas: 1
  selector:
    matchLabels:
      app: biblioteca
  template:
    metadata:
      labels:
        app: biblioteca
    spec:
      containers:
        - name: biblioteca
          image: IMAGEM_DO_REGISTRO/biblioteca:1.0
          imagePullPolicy: IfNotPresent
          ports:
            - containerPort: 8080
          env:
            - name: API_SECURITY_TOKEN_SECRET
              valueFrom:
                secretKeyRef:
                  name: biblioteca-secret
                  key: jwt-secret
      resources:
        requests:
          cpu: "250m"
          memory: "384Mi"
        limits:
          cpu: "1"
          memory: "768Mi"
      readinessProbe:
        httpGet:
          path: /login
          port: 8080
          initialDelaySeconds: 20
          periodSeconds: 10
      livenessProbe:
        httpGet:
          path: /login
          port: 8080
          initialDelaySeconds: 40
          periodSeconds: 20

```

GKE Autopilot: O Autopilot pode elevar automaticamente requests abaixo dos mínimos da plataforma. Depois do deploy, registre os recursos efetivamente aplicados ao Pod. Se o cluster não suportar bursting, 384Mi pode ser ajustado para pelo menos 512Mi.

8.3 Service para AWS e Google Cloud

k8s/service-cloud.yaml:

```

apiVersion: v1
kind: Service
metadata:
  name: biblioteca
spec:
  type: LoadBalancer
  selector:
    app: biblioteca
  ports:
    - port: 80
      targetPort: 8080

```

8.4 Service para VPS

k8s/service-vps.yaml:

```
apiVersion: v1
kind: Service
metadata:
  name: biblioteca
spec:
  type: NodePort
  selector:
    app: biblioteca
  ports:
    - port: 8080
      targetPort: 8080
      nodePort: 30080
```

8.5 Comandos de verificação

```
kubectl get nodes
kubectl get pods -o wide
kubectl get deployment
kubectl get service
kubectl rollout status deployment/biblioteca
kubectl describe pod NOME_DO_POD
kubectl logs deployment/biblioteca
```

9. Provedor 1 - AWS com Amazon EKS e ECR

Configuração usada na comparação: Região us-east-1; EKS em suporte padrão; 1 node gerenciado t3.medium (2 vCPU, 4 GiB); ECR para imagem; Service LoadBalancer.

9.1 Autenticar a AWS CLI

```
aws configure
aws sts get-caller-identity
```

A saída de aws sts get-caller-identity deve mostrar a conta e a identidade IAM em uso. Não coloque chaves de acesso no documento ou em prints.

9.2 Criar o repositório ECR

```
aws ecr create-repository --repository-name biblioteca --region us-east-1

# Linux/macOS/Git Bash
ACCOUNT_ID=$(aws sts get-caller-identity --query Account --output text)
aws ecr get-login-password --region us-east-1 | \
  docker login --username AWS --password-stdin \
    ${ACCOUNT_ID}.dkr.ecr.us-east-1.amazonaws.com

docker tag biblioteca:1.0 \
  ${ACCOUNT_ID}.dkr.ecr.us-east-1.amazonaws.com/biblioteca:1.0
docker push ${ACCOUNT_ID}.dkr.ecr.us-east-1.amazonaws.com/biblioteca:1.0

# Windows PowerShell
$ACCOUNT_ID = aws sts get-caller-identity --query Account --output text
aws ecr get-login-password --region us-east-1 | docker login --username AWS --password-stdin
"$ACCOUNT_ID.dkr.ecr.us-east-1.amazonaws.com"
docker tag biblioteca:1.0 "$ACCOUNT_ID.dkr.ecr.us-east-1.amazonaws.com/biblioteca:1.0"
docker push "$ACCOUNT_ID.dkr.ecr.us-east-1.amazonaws.com/biblioteca:1.0"
```

9.3 Criar o cluster EKS com um node

```
eksctl create cluster \
  --name biblioteca \
  --region us-east-1 \
  --managed \
  --nodegroup-name ng-biblioteca \
  --node-type t3.medium \
  --nodes 1 \
  --nodes-min 1 \
  --nodes-max 1
```

Atenção a custos: O EKS cobra pelo cluster e também pelos recursos usados pelos nodes, discos, registros e balanceadores. Exclua o laboratório ao terminar.

Confirmar o contexto:

```
aws eks update-kubeconfig --region us-east-1 --name biblioteca
kubectl get nodes
```

9.4 Ajustar e aplicar os manifestos

No arquivo k8s/deployment.yaml, substitua IMAGEM_DO_REGISTRO/biblioteca:1.0 pelo endereço completo do ECR, por exemplo:

123456789012.dkr.ecr.us-east-1.amazonaws.com/biblioteca:1.0.

```
kubectl create secret generic biblioteca-secret \
  --from-literal=jwt-secret="VALOR_FORTE_E_NAO_PUBLICADO"

kubectl apply -f k8s/deployment.yaml
kubectl apply -f k8s/service-cloud.yaml
kubectl rollout status deployment/biblioteca
kubectl get pods
kubectl get service biblioteca
```

Aguarde até a coluna EXTERNAL-IP/hostname do Service deixar de aparecer como <pending>. Copie o endereço público e abra http://ENDereco_PUBLICO/login.

9.5 Validação AWS

- Cluster EKS aparece como ativo e o node está Ready.
- Imagem biblioteca:1.0 aparece no ECR.
- Deployment biblioteca está Available com 1 réplica.
- Pod está Running e Ready 1/1.
- Service possui endereço externo.
- Login admin/admin funciona.
- Cadastro/consulta de autores e livros funciona durante a vida do Pod.
- kubectl logs deployment/biblioteca não apresenta erro impeditivo.

9.6 Limpeza AWS

```
kubectl delete -f k8s/service-cloud.yaml
kubectl delete -f k8s/deployment.yaml
kubectl delete secret biblioteca-secret

eksctl delete cluster --name biblioteca --region us-east-1
aws ecr delete-repository --repository-name biblioteca --region us-east-1 --force
```

Verificação final: Depois da exclusão, confira no Console AWS se não ficaram Load Balancers, volumes EBS, Elastic IPs ou stacks do CloudFormation associados ao laboratório.

10. Provedor 2 - Google Cloud com GKE Autopilot e Artifact Registry

Configuração usada na comparação: Região us-central1; GKE Autopilot; 1 réplica; Artifact Registry; Service LoadBalancer. O free tier de gerenciamento do GKE é apresentado separadamente do preço normal.

10.1 Autenticar, escolher projeto e habilitar APIs

```
gcloud auth login
gcloud config set project ID_DO_PROJETO
gcloud services enable container.googleapis.com artifactregistry.googleapis.com
```

10.2 Criar Artifact Registry e publicar a imagem

```
gcloud artifacts repositories create biblioteca \
  --repository-format=docker \
  --location=us-central1 \
  --description="Imagens do projeto Biblioteca"

gcloud auth configure-docker us-central1-docker.pkg.dev

docker tag biblioteca:1.0 \
  us-central1-docker.pkg.dev/ID_DO_PROJETO/biblioteca/biblioteca:1.0

docker push \
  us-central1-docker.pkg.dev/ID_DO_PROJETO/biblioteca/biblioteca:1.0
```

10.3 Criar cluster GKE Autopilot

```
gcloud container clusters create-auto biblioteca --region us-central1
gcloud container clusters get-credentials biblioteca --region us-central1
kubectl get nodes
```

No Autopilot, o Google gerencia os nodes e provisiona capacidade conforme os workloads. Por isso, a comparação registra os recursos efetivos do Pod em vez de um tipo fixo de VM.

10.4 Ajustar e aplicar os manifestos

No deployment.yaml, use us-central1-docker.pkg.dev/ID_DO_PROJETO/biblioteca/biblioteca:1.0 no campo image.

```
kubectl create secret generic biblioteca-secret \
  --from-literal=jwt-secret="VALOR_FORTE_E_NAO_PUBLICADO"

kubectl apply -f k8s/deployment.yaml
kubectl apply -f k8s/service-cloud.yaml
kubectl rollout status deployment/biblioteca
kubectl get pods
kubectl get service biblioteca
```

Registrar requests efetivamente aplicados (Linux/Git Bash):

```
kubectl get pod -l app=biblioteca -o yaml | grep -A 12 resources:
```

No Windows PowerShell, use `kubectl get pod -l app=biblioteca -o yaml` e procure a seção `resources`. O Autopilot pode ajustar requests mínimos.

10.5 Validação GCP

- Cluster Autopilot está em execução.
- Imagem está no Artifact Registry.
- Pod está Running e Ready.
- Service possui EXTERNAL-IP.
- Página /login abre pelo IP externo.
- Login e operações web funcionam.
- API /api/auth/login retorna JWT.
- Logs não apresentam falha impeditiva.

10.6 Limpeza GCP

```
kubectl delete -f k8s/service-cloud.yaml
kubectl delete -f k8s/deployment.yaml
kubectl delete secret biblioteca-secret

gcloud container clusters delete biblioteca --region us-central1
gcloud artifacts repositories delete biblioteca --location us-central1
```

Verificação final: Confirme no Console Google Cloud se o cluster foi removido e se não restaram regras de encaminhamento, endereços IP ou recursos de balanceamento de carga cobrados.

11. Provedor 3 - DigitalOcean VPS com K3s

Configuração usada na comparação: Droplet Basic com 2 vCPU, 4 GiB de RAM e 80 GiB SSD, Ubuntu LTS, região nyc3 (ou equivalente disponível). K3s de nó único e NodePort 30080.

11.1 Criar a VPS

1. No painel da DigitalOcean, clique em Create > Droplets.
2. Selecione Ubuntu LTS x64.
3. Escolha uma região disponível, registrando-a no trabalho; para esta comparação, considere nyc3.
4. Escolha Basic, 2 vCPU e 4 GiB de RAM (80 GiB SSD no plano exibido no preço consultado).
5. Use autenticação por chave SSH, preferencialmente, e crie o Droplet.
6. Registre o IPv4 público e não publique chaves privadas.

Conectar por SSH:

```
ssh root@IP_DA_VPS
```

11.2 Atualizar o sistema e instalar K3s

```
apt update && apt upgrade -y
curl -sfl https://get.k3s.io | sh -
sudo kubectl get nodes
```

A instalação de nó único do K3s já fornece um cluster Kubernetes funcional, incluindo control plane, kubelet e runtime de containers. O kubeconfig local do servidor fica em /etc/rancher/k3s/k3s.yaml.

11.3 Liberar a porta de teste

No firewall da DigitalOcean, permita SSH (TCP 22) somente a partir do IP administrativo quando possível e permita TCP 30080 para o acesso acadêmico à aplicação. Se usar UFW dentro da VPS, aplique também as regras locais.

Exemplo com UFW:

```
ufw allow OpenSSH
ufw allow 30080/tcp
ufw enable
ufw status
```

Segurança: Expor NodePort diretamente é adequado apenas ao laboratório. Em produção, prefira um Ingress/proxy reverso com HTTPS e controle de firewall mais restritivo.

11.4 Fazer os arquivos chegarem à VPS

Opção A - clonar o repositório na VPS e copiar a pasta k8s preparada. Opção B - enviar apenas a pasta k8s por SCP. Como a imagem já está em um registro acessível, não é necessário compilar Java ou Docker na VPS.

Enviar manifests via SCP:

```
# No computador local
scp -r k8s root@IP_DA_VPS:/root/
```

11.5 Definir a imagem na VPS

Para preservar a mesma imagem, use uma referência pública acessível pela VPS. A forma mais simples é publicar a mesma imagem biblioteca:1.0 em Docker Hub ou tornar temporariamente público um registro usado no laboratório. Se o registro for privado, configure imagePullSecret.

Alternativa recomendada para a atividade: Publicar a mesma imagem também em Docker Hub com uma tag imutável simplifica o pull na VPS. Registre o digest para demonstrar que o artefato corresponde à mesma imagem construída localmente.

Executar no computador local, se usar Docker Hub:

```
docker tag biblioteca:1.0 SEU_USUARIO_DOCKERHUB/biblioteca:1.0
docker login
docker push SEU_USUARIO_DOCKERHUB/biblioteca:1.0
```

Edite /root/k8s/deployment.yaml na VPS e substitua o campo image por SEU_USUARIO_DOCKERHUB/biblioteca:1.0.

11.6 Implantar no K3s

```
sudo kubectl create secret generic biblioteca-secret \
  --from-literal=jwt-secret="VALOR_FORTE_E_NAO_PUBLICADO"

sudo kubectl apply -f /root/k8s/deployment.yaml
sudo kubectl apply -f /root/k8s/service-vps.yaml
sudo kubectl rollout status deployment/biblioteca
sudo kubectl get pods -o wide
sudo kubectl get services
```

Acesse http://IP_DA_VPS:30080/login e realize os testes funcionais. Se a conexão falhar, verifique o firewall da DigitalOcean, UFW, o Service NodePort e o estado Ready do Pod.

11.7 Validação VPS

- Droplet ativo, com IP público registrado.
- K3s instalado e node em Ready.
- Deployment com 1 réplica disponível.
- Pod Running e Ready 1/1.
- Service NodePort mostra 30080/TCP.
- Porta 30080 liberada no firewall.
- Página de login acessível externamente.
- Login e operações funcionam.
- Logs sem erro impeditivo.

11.8 HTTPS em um cenário de produção

Para produção, uma evolução seria instalar um Ingress Controller ou usar o Traefik já distribuído com K3s, apontar um domínio para o IP público e emitir certificado TLS, por exemplo via ACME/Let's Encrypt. Isso não é obrigatório para o laboratório, mas deve ser citado como melhoria.

11.9 Limpeza da VPS

```
sudo kubectl delete -f /root/k8s/service-vps.yaml
sudo kubectl delete -f /root/k8s/deployment.yaml
sudo kubectl delete secret biblioteca-secret
/usr/local/bin/k3s-uninstall.sh
```

Depois, exclua o Droplet e qualquer Firewall/Volume reservado para a atividade no painel da DigitalOcean para interromper cobranças.

12. Validação funcional e evidências

12.1 Testes funcionais mínimos por ambiente

CT	Teste	Passos resumidos	Resultado esperado
CT01	Acesso à página de login	Abrir /login pelo endereço público	Página carrega sem erro 5xx
CT02	Login administrador	Entrar com admin/admin	Acesso concedido com perfil ADMIN
CT03	Cadastro de autor	Cadastrar um autor pela interface	Registro aparece na consulta enquanto o Pod permanece ativo
CT04	Cadastro de livro	Cadastrar um livro e vinculá-lo conforme interface	Livro aparece na consulta

CT05	Login REST	POST /api/auth/login com admin/admin	Resposta retorna token JWT
CT06	Bearer Token	Chamar rota protegida com Authorization: Bearer <token>	Acesso autenticado
CT07	Logs	Consultar logs do Deployment	Sem erro impeditivo
CT08	Limitação H2	Reiniciar o Pod e consultar dados criados	Dados transitórios podem ser perdidos; limitação registrada

12.2 Evidências obrigatórias por ambiente

- Nome do provedor, produto e região.
- Tela/comando mostrando cluster e nodes disponíveis.
- Imagem publicada no registro escolhido.
- Deployment disponível.
- Pod Running e Ready.
- Service ou mecanismo de exposição.
- Aplicação acessível externamente.
- Login realizado com sucesso.
- Logs sem erro impeditivo.
- Procedimento de remoção dos recursos.

Privacidade: Ocultar tokens, chaves, segredos, IDs sensíveis e informações pessoais de cobrança nas capturas de tela.

12.3 Modelo para colar as evidências

AWS - EKS

Evidência	Captura / observação
Cluster/nó	[INSERIR CAPTURA]
Imagem no registro	[INSERIR CAPTURA]
Pod e Deployment	[INSERIR CAPTURA]
Service/endereço	[INSERIR CAPTURA]
Login funcional	[INSERIR CAPTURA]
Logs	[INSERIR CAPTURA]

Google Cloud - GKE

Evidência	Captura / observação
Cluster/nó	[INSERIR CAPTURA]
Imagem no registro	[INSERIR CAPTURA]
Pod e Deployment	[INSERIR CAPTURA]
Service/endereço	[INSERIR CAPTURA]
Login funcional	[INSERIR CAPTURA]
Logs	[INSERIR CAPTURA]

DigitalOcean - K3s

Evidência	Captura / observação
Cluster/nó	[INSERIR CAPTURA]
Imagem no registro	[INSERIR CAPTURA]
Pod e Deployment	[INSERIR CAPTURA]
Service/endereço	[INSERIR CAPTURA]
Login funcional	[INSERIR CAPTURA]
Logs	[INSERIR CAPTURA]

13. Diagnóstico e solução de erros comuns

Sintoma	Comandos de diagnóstico	Causa provável / ação
Pod em ImagePullBackOff	kubectl describe pod NOME	Imagem/endereço incorreto, registro privado, autenticação ausente. Corrigir image ou imagePullSecret.
Pod em CrashLoopBackOff	kubectl logs deployment/biblioteca; kubectl describe pod NOME	Erro no startup, segredo JWT inválido/ausente, memória insuficiente.
Readiness 0/1	kubectl describe pod NOME; kubectl logs deployment/biblioteca	/login ainda não responde ou aplicação demorou para iniciar. Ajustar delays apenas após verificar logs.
Service LoadBalancer em pending	kubectl describe service biblioteca	Provisionamento do provedor, quota, permissões ou integração de rede ainda não concluída.
VPS não abre 30080	sudo kubectl get svc; sudo ss -ltnp; ufw status	Firewall externo/UFW bloqueando; IP incorreto; Pod não Ready.
401 na API	Repetir POST /api/auth/login e inspecionar cabeçalho	Token ausente/expirado ou Bearer formatado incorretamente.
Dados sumiram após restart	kubectl get pods; histórico do rollout	Comportamento esperado do H2 em memória. Não tratar como falha do Kubernetes.
Build Docker falha no COPY do JAR	docker build --progress=plain; verificar target	Nome/versionamento do JAR diferente do Dockerfile. Ajustar o COPY.

Comandos adicionais úteis:

```
kubectl get events --sort-by=.lastTimestamp
kubectl describe deployment biblioteca
kubectl describe service biblioteca
kubectl logs deployment/biblioteca --previous
```

14. Comparação de custos

Estimativa consultada em 02/09/2026, em USD, para aproximadamente 730 horas/mês. Valores de tráfego, impostos, variações regionais, LCUs/GB processados e recursos adicionais podem alterar a fatura. Créditos gratuitos são apresentados separadamente.

Critério	AWS	Google Cloud	DigitalOcean VPS
Serviço Kubernetes	Amazon EKS	GKE Autopilot	K3s em VPS
Região/configuração	us-east-1; 1x t3.medium	us-central1; 1 Pod; 250m CPU; 512Mi memória; 1Gi ephemeral p/ estimativa	nyc3; 2 vCPU; 4GiB; 80GiB
Gestão do cluster / mês	US\$ 73.00	US\$ 73.00 (antes do free tier)	US\$ 0,00 (software K3s)

Computação / mês	US\$ 30.37	US\$ 10.02 aprox.	Incluída no Droplet
Disco/armazenamento	US\$ 1.60 aprox. (20GiB gp3 assumidos)	Incluído no modelo de cobrança do Pod / Artifact Registry até 0,5GiB sem custo	Incluído: 80GiB SSD
Registro de imagem	US\$ 0.05 para ~0,5GB	US\$ 0,00 até 0,5GiB/mês	Docker Hub público: US\$ 0,00 no cenário acadêmico
Balanceador/IP/rede	US\$ 16.43 base de ALB; uso/LCU à parte	US\$ 18.25 para regra de encaminhamento; processamento à parte	Sem LB separado: NodePort no IP do Droplet
Total mensal normal	US\$ 121.44 + uso variável	US\$ 101.27 + uso variável	US\$ 24.00
Total com crédito/free tier aplicável	Não considerado	US\$ 28.27 aprox. se o crédito mensal de US\$ 74,40 cobrir a taxa do cluster	Não considerado

Leitura correta dos preços: No GKE, o free tier oferece crédito mensal equivalente a um cluster Autopilot ou zonal, mas esse crédito não paga computação do workload. No relatório, mostre sempre o valor normal e, em linha separada, o efeito do crédito.

AWS Load Balancer: O valor de US\$ 0,0225/h é uma referência de cobrança horária do Application Load Balancer em us-east-1; LCUs e tráfego são cobrados à parte. O tipo efetivamente provisionado pelo Service e seu custo devem ser confirmados na conta usada no laboratório.

Estimativa, não fatura: A tabela é uma estimativa acadêmica baseada em preços públicos e hipóteses explícitas. A fatura real pode variar. Atualize a data e os valores se o trabalho for entregue em outra data.

15. Comparação qualitativa

Critério	AWS EKS	Google GKE Autopilot	DigitalOcean VPS + K3s
Facilidade de configuração	Média: várias integrações e IAM	Alta: Autopilot reduz gestão de nodes	Média: VPS simples, mas K3s/firewall são responsabilidade da equipe
Kubernetes gerenciado	Sim	Sim	Não; K3s é administrado pela equipe
Escalabilidade	Alta	Alta e muito automatizada	Limitada ao tamanho/número de VPSs e à administração manual
Integração com registro	Excelente com ECR	Excelente com Artifact Registry	Externa, por exemplo Docker Hub
Observabilidade	CloudWatch e integrações AWS	Cloud Logging/Monitoring integrados	Exige configuração própria ou ferramentas adicionais
Responsabilidade operacional	Baixa no control plane; média nos nodes e integrações	Baixa para infraestrutura dos nodes	Alta: SO, patches, K3s, firewall, backup e disponibilidade
Vantagem principal	Ecossistema amplo e integração corporativa	Menor esforço operacional para o workload	Custo previsível e controle direto
Desvantagem principal	Custo e curva de IAM/rede	Custos de LB/recursos e dependência do provedor	Sem HA por padrão; equipe assume segurança e manutenção
Cenário recomendado	Ambientes corporativos já integrados à AWS	Times que querem Kubernetes com mínima gestão de nodes	Laboratórios, protótipos e serviços pequenos com equipe capaz de operar Linux/Kubernetes

16. Recomendação final

Para este projeto acadêmico, a DigitalOcean VPS com K3s tende a apresentar o menor custo mensal direto e uma arquitetura fácil de compreender, mas transfere à equipe a responsabilidade por segurança, atualização e disponibilidade. Para um cenário profissional que priorize menor esforço operacional, o GKE Autopilot oferece uma experiência mais simples porque o provedor gerencia os nodes e ajusta capacidade conforme o workload. O Amazon EKS é uma excelente opção em organizações já padronizadas no ecossistema AWS, porém, para uma aplicação pequena com apenas uma réplica e H2 em memória, seu custo fixo de cluster pode ser desproporcional.

Assim, a recomendação deste trabalho é: GKE Autopilot para um cenário de nuvem gerenciada com foco em simplicidade operacional; DigitalOcean + K3s para laboratório e economia; EKS quando integração com serviços AWS, governança e padronização corporativa justificarem o custo adicional.

Limitação que muda tudo: A aplicação atual não é stateless porque mantém o H2 em memória. Antes de escalar horizontalmente ou buscar alta disponibilidade, a prioridade técnica deve ser externalizar a persistência para um banco durável.

17. Limpeza dos recursos

A limpeza é parte obrigatória do tutorial porque clusters, VMs, discos, registros, IPs e balanceadores podem continuar gerando cobrança mesmo sem uso ativo da aplicação.

Ambiente	O que remover	Como confirmar
AWS	Service/Deployment/Secret, cluster EKS, nodegroup/EC2, ECR, LB, volumes/stacks remanescentes	Console EKS, EC2, ECR, Load Balancers e CloudFormation sem recursos do laboratório
Google Cloud	Service/Deployment/Secret, cluster GKE, Artifact Registry, LB/forwarding rule/IP remanescente	Console GKE, Artifact Registry, Network Services e IP addresses
DigitalOcean	Objetos K3s, K3s, Droplet, firewall/volume dedicado	Painel DigitalOcean sem Droplet e recursos associados

18. Referências oficiais

Projeto Biblioteca - repositório oficial do exercício. <https://github.com/dpRanghetti/biblioteca>

Docker Docs - Multi-stage builds. <https://docs.docker.com/build/building/multi-stage/>

Kubernetes - Deployments. <https://kubernetes.io/docs/concepts/workloads/controllers/deployment/>

Kubernetes - Services. <https://kubernetes.io/docs/concepts/services-networking/service/>

Kubernetes - Secrets. <https://kubernetes.io/docs/concepts/configuration/secret/>

Kubernetes - Liveness, Readiness e Startup Probes.

<https://kubernetes.io/docs/tasks/configure-pod-container/configure-liveness-readiness-startup-probes/>

AWS - Create an Amazon EKS cluster.

<https://docs.aws.amazon.com/eks/latest/userguide/create-cluster.html>

AWS - Get started with EKS using eksctl.

<https://docs.aws.amazon.com/eks/latest/userguide/getting-started-eksctl.html>

AWS - Amazon EKS Pricing. <https://aws.amazon.com/eks/pricing/>

AWS - Amazon ECR Pricing. <https://aws.amazon.com/ecr/pricing/>

AWS CLI - ECR get-login-password.

<https://docs.aws.amazon.com/cli/latest/reference/ecr/get-login-password.html>

AWS - Elastic Load Balancing Pricing. <https://aws.amazon.com/elasticloadbalancing/pricing/>

Google Cloud - Create an Autopilot cluster.

<https://cloud.google.com/kubernetes-engine/docs/how-to/creating-an-autopilot-cluster>

Google Cloud - GKE Pricing. <https://cloud.google.com/kubernetes-engine/pricing>

Google Cloud - Autopilot resource requests.

<https://cloud.google.com/kubernetes-engine/docs/concepts/autopilot-resource-requests>

Google Cloud - Create Artifact Registry repository.

<https://cloud.google.com/artifact-registry/docs/repositories/create-repos>

Google Cloud - Authenticate Docker to Artifact Registry.

<https://cloud.google.com/artifact-registry/docs/docker/authentication>

Google Cloud - Artifact Registry Pricing. <https://cloud.google.com/artifact-registry/pricing>

Google Cloud - Load Balancing Pricing. <https://cloud.google.com/load-balancing/pricing>

K3s - Quick-Start Guide. <https://docs.k3s.io/quick-start>

DigitalOcean - Create a Droplet. <https://docs.digitalocean.com/products/droplets/how-to/create/>

DigitalOcean - Droplet Pricing. <https://www.digitalocean.com/pricing/droplets>

Apêndice A - Sequência resumida de execução

Use este apêndice apenas como lembrete. O passo a passo detalhado e as observações de segurança permanecem nas seções anteriores.

```
# 1) Clonar
git clone https://github.com/dpRanghetti/biblioteca.git
cd biblioteca

# 2) Criar Dockerfile e .dockerignore
# 3) Build
docker build -t biblioteca:1.0 .

# 4) Testar local
docker run --name biblioteca-local -p 8080:8080 -e API_SECURITY_TOKEN_SECRET="SEGredo"
biblioteca:1.0

# 5) Validar http://localhost:8080/login
# 6) Publicar a mesma imagem nos registros escolhidos
# 7) Criar cluster
# 8) Criar Secret Kubernetes
# 9) Aplicar Deployment + Service
# 10) Validar Pod, Service, login e logs
# 11) Registrar evidências e custos
# 12) Remover todos os recursos
```